Red del barrio

Red de capacidades y confianza vecinal

Contexto completo del proyecto, de inicio a fin

VIII Hackathon UTP - El Comercio 2026

Documento generado el 8 de noviembre de 2026 a partir del repositorio Jorge-Luis22/red-del-barrio (todas sus ramas) y de la carpeta local Proyecto_Hackaton_Comercio_2026 (rama mobile-ux).

Nota: lo que sigue sale de leer el codigo y los documentos. Solo se ejecuto la PWA de main (en el navegador) y la app Flutter de feat/flutter-mobile (hasta su pantalla de entrada). Nada mas fue probado.

1. Resumen en una pagina

Que es. Una app barrial: el vecino dice que necesita, su barrio propone como resolverlo, cada interaccion confirmada construye reputacion, y esa misma red se activa en una emergencia.

Para que evento. Hackathon UTP-El Comercio 2026 (virtual, 48 horas, del viernes 6 al domingo 8 de noviembre). El jurado evalua lo que ve funcionando.

Quien. Equipo de Jorge (Daniel, Adams, Andre, Jose). El lider/integracion es Jorge.

Donde esta hoy. Existen cuatro lineas de trabajo que conviven:

- **Simulador / PWA de un solo archivo** (index.html, ~1.900 lineas, sin librerias): la demo para el jurado.
- **App React + TypeScript** (carpeta app/): SQLite local, Capacitor, Supabase y Matrix. Es el piloto web.
- **App Flutter del equipo** (rama feat/flutter-mobile, carpeta mobile/): version movil conectada al mismo Supabase del piloto.
- **App Flutter de Jose** (rama mobile-ux en tu carpeta local): reescritura movil con arquitectura hexagonal, Drift y trabajo sin servidor.

La regla que ordena todo: la demo nunca se cae. Si Matrix falla, el chat sigue local; si Supabase falla, SQLite sigue respondiendo; si el APK falla, se presenta la PWA instalada. Nunca una pantalla en blanco.

2. El reto y los criterios

Texto de la organizacion. Crear una aplicacion para que los ciudadanos obtengan, con costos minimos y sin transporte mayor, productos y servicios avalados por recomendaciones de vecinos. Debe integrar intercambios solidarios y evolucionar en emergencias para coordinar soporte.

Caracteristicas funcionales pedidas

- Informacion recomendada de productos y servicios cercanos a la zona de residencia.
- Facilidad para promover servicios y compartir recomendaciones vecinales.
- Facilidad para solicitar y ofrecer ayuda e intercambios solidarios.
- Identificar servicios y grupos de apoyo disponibles en emergencias.
- Coordinar necesidades de ayuda y recursos entre vecinos en emergencias.
- Geolocalizar servicios, recursos y necesidades de apoyo.
- Tutorial de uso.
- Enfoque movil, con maximo tres clics para publicar.
- Funcionamiento online y offline.
- Uso intuitivo y accesible para cualquier vecino.

Modulos

- **Modulo 1, servicios de proximidad:** directorio de oficios validado con el "Visto Bueno Vecinal"; en emergencias, directorio de servicios y grupos de apoyo.
- **Modulo 2, cadena de favores:** muro colaborativo para resolver necesidades sin dinero (prestamos, ayuda temporal, donaciones, intercambio de habilidades, movilidad solidaria); en emergencias, tablero de ayuda con reportes geolocalizados.

Fechas

Hito	Dato
Inicio del desarrollo	Viernes 6 de noviembre, 7:00 p. m.
Cierre asumido	Domingo 8, 6:30 p. m. (hora de Lima), sin confirmar con la organizacion
Propuestas	10, 11 o 12 de noviembre, 7:00 p. m. segun sorteo: 15 min de exposicion y 5 de preguntas
Finalistas	Se anuncian el 13; presentan el 17 de noviembre, 6:00 a 8:30 p. m.: 5 min de presentacion y 15 de demo
Premios	1.o USD 2,000 y difusion en El Comercio; 2.o USD 800

Criterios del jurado (sin pesos publicados): funcionalidad (usabilidad, confiabilidad, escalabilidad, propuesta, codigo y programa, innovacion) y presentacion (comunicacion, manejo del tiempo, recursos audiovisuales, organizacion, respuesta a preguntas). Cuatro criterios premian que lo mostrado funcione y sea estable.

3. El producto

Insight. Lo cotidiano construye la red que se usa en emergencias: quien ya se conoce y se ha ayudado responde mejor cuando algo grave pasa.

Diferencial. Facebook organiza publicaciones, WhatsApp conversaciones y los marketplaces anuncios; esta app organiza necesidades y su resolucion.

Reputacion contextual. Ser buen gasfitero no es lo mismo que ser buen vecino prestador: cada contexto tiene su propia nota y solo cuentan interacciones confirmadas.

Usuario objetivo. Vecinos de alrededor de 30 anos, con foco en simplicidad.

Flujo principal

- **Pedir.** El vecino escribe o elige que necesita (servicio, producto, prestamo o ayuda). En "Explorar cerca" puede pedir un servicio en 2 toques.
- **Proponer.** El proveedor no publica precio de antemano: cuenta que hace y cuando esta disponible; ante un pedido propone su precio.
- **Aceptar y encontrarse.** El vecino acepta una propuesta, se genera un codigo de 4 digitos para verificar el encuentro y se abre un chat.
- **Ajuste de precio.** Si el servicio cambia en el lugar, el proveedor propone un nuevo precio con motivo y foto; solo cambia si el vecino acepta. Sin contraofertas.
- **Cerrar.** Calificacion en ambos sentidos; alimenta la reputacion y el Visto Bueno.

Reputacion y nivel solidario

Hay reputacion de servicio (estrellas) y reputacion entre vecinos, por separado. "Tu ayuda al barrio" es un resumen de evidencia (ayudas confirmadas, personas distintas, prestamos completados, tiempo aportado, calificacion promedio). **No hay puntos, puestos ni prioridad.** Anti-abuso: una ayuda cuenta solo si ambos confirman, con maximo 2 con la misma persona.

Emergencias

- Tipos: sismo, incendio, medico, persona mayor en riesgo y violencia (esta ultima es privada, no sale en el muro).
- **Primero la institucion oficial**, despues la red vecinal.
- La ubicacion exacta nunca es publica: se ve una zona aproximada; la exacta la recibe solo quien acepto ayudar.
- Brigadista vecinal voluntario (sin verificar); cualquier vecino puede proponer un punto de acopio, marcado como "sin verificar".
- La app avisa a vecinos; no reemplaza a la PNP (105), Bomberos (116) ni al SAMU (106).

Reglas que no se pueden romper

Maximo 3 clics en las acciones principales; funciona sin internet; la ubicacion exacta nunca es publica; los puntos y puestos de reputacion no son publicos; en emergencia siempre va primero la institucion oficial; texto en espanol claro y sencillo.

Que no es el producto

No reemplaza servicios oficiales de emergencia, no procesa pagos en esta version y no promete comunicacion sin red entre celulares. La verificacion de identidad (RENIEC), los pagos y el seguimiento en

vivo estan como "Proximamente".

4. Historia y ramas del repositorio

Repositorio del equipo: github.com/Jorge-Luis22/red-del-barrio. Tu copia local tiene como remoto JoshRivas313/Proyecto_Hackaton_Comercio_2026. Los commits iniciales (xxxx, ## Bases, tecnico, "Agrega la app Red del barrio (PWA)") son el punto comun de todas las lineas.

Rama	Tipo	Que aporta
main (Jorge-Luis22)	PWA	Simulador + app React (app/), infra Synapse/Caddy, docs, Supabase. Ultimo commit "New public".
feat/gps-trato-calificacion	PWA	Tutorial, accesibilidad, directorio de oficios, nivel calculado, aviso de seguridad, muro de emergencia entre telefonos, cadena de favores sin dinero, recursos criticos, mapa del barrio.
feat/piloto-real	PWA	Piloto con vecinos reales: cuentas anonimas automaticas + codigo del barrio, chats privados, reportar/bloquear, eliminar cuenta, migraciones Supabase 0001-0004, docs/PILOTO.md.
feat/publicar-3-toques	PWA	Publicar en 3 toques: hoja con botones grandes (Necesito/Ofrezco, rubro, Publicar), cuadro de texto libre.
feat/flutter-mobile	**Movil (Flutter)**	Carpeta mobile/ con 18 archivos .dart y CI de GitHub (mobile-apk.yml) que compila el APK. Mismo servidor Supabase del piloto.
mobile-ux (local, tuya)	**Movil (Flutter)**	Reescritura propia en la raiz: arquitectura hexagonal, Drift, cola de pendientes, 59 archivos .dart, 27 pruebas. Sin servidor.
main (local)	-	Atrasada 4 commits respecto a origin/main; solo trae la PWA original. origin/main local ya incluye Supabase (client_stats privada).

Respuesta corta a "cual es la version movil, no la PWA": feat/flutter-mobile en el repositorio de Jorge, y mobile-ux en tu carpeta local. Las demas ramas son PWA.

Nota: el DECISIONES.md de main descarta Flutter a favor de React, pero esta desactualizado: ramas posteriores si incluyen Flutter.

5. La PWA y la app React (main)

5.1 El simulador / PWA de un archivo

- index.html (~1.876 lineas), sin librerias; sw.js para offline; manifest e iconos.
- Guarda datos en localStorage (clave redvecinal.v8), usa GPS real y funciona sin internet.
- Idiomas: espanol completo; ingles y quechua en pantallas principales (quechua sin revision nativa).
- Es la "especificacion" del producto: si algo no esta en el simulador, no se construye.

5.2 La app React (carpeta app/)

- Stack: React 18, TypeScript, Vite 6, Capacitor 6 (PWA + APK Android), sql.js (SQLite WASM) en IndexedDB.
- Toda la UI habla con db/repo.ts (716 lineas), la unica puerta a los datos.
- Cola offline outbox.ts con txn_id idempotente; chat por Matrix/Synapse con wrapper REST propio (modo local si no hay servidor).
- Servidor previsto: Supabase (Postgres, RLS, Realtime, Storage). Infra: docker compose con Synapse + Caddy (~USD 5/mes).
- Codigo de encuentro de 4 digitos guardado con hash, nunca en el chat. Ubicacion publica redondeada.

5.3 Piloto real (feat/piloto-real)

	Demo (jurado)	Piloto (vecinos)
Entrada	Abre directo, vecinos de ejemplo	Codigo del barrio + nombre + zona (18+)
Cuentas	No hay	Una por telefono, automatica (anonima), sin correo
Datos de ejemplo	Si	Ninguno
Chats	Tabla abierta	Solo los dos del trato (lo exige el servidor)
Seguridad	-	Reportar, bloquear, eliminar mi cuenta, terminos

Se compila con VITE_APP_MODE=pilot. Direccion del piloto: dynamic-bunny-001690.netlify.app (8-nov-2026). El codigo del barrio NO esta en el repositorio: se fija en Supabase con pilot_set_code. Limites declarados: cuenta anclada al telefono, sin notificaciones push, sin cifrado de extremo a extremo, un vetado puede crear otra cuenta, altas limitadas por IP, y los datos personales (Ley 29733) merecen revision legal antes de abrirlo mas.

6. Las dos apps moviles (Flutter)

6.1 feat/flutter-mobile (equipo de Jorge)

- Carpeta mobile/: api.dart (cuenta anonima + llamadas a Supabase), store.dart (datos en memoria y sync), catalog.dart (botones de oficios), theme.dart y pantallas: onboarding, wall, directory, need_detail, publish_sheet, chat, messages, profile, moderation, legal, home.
- La carpeta android/ no se guarda: el flujo de GitHub la genera con flutter create. APK por Actions, artefacto red-del-barrio-apk.
- Incluye: entrar con codigo del piloto, muro con busqueda y filtros, publicar por botones y texto libre, propuestas, codigo de encuentro, chat, calificar, directorio con recomendaciones, perfil con nivel, bloqueo, reporte y borrar cuenta.
- Todavia no incluye: Emergencia, GPS, favores sin dinero y recursos, ni guardado offline entre aperturas.
- **Verificado en esta sesion:** flutter analyze lib sin errores; APK de depuracion compilado e instalado en un emulador Android 14; la app abre en la pantalla de entrada (pide codigo del piloto, nombre y zona). No se entro: requiere el codigo y crearia una cuenta anonima en el Supabase real de Jorge.

6.2 mobile-ux (tu rama local)

- Fase 1 (analisis): docs 00-contexto, 00-benchmark, 01-analisis, 02-demo, 03-riesgos, 04-arquitectura, decisiones y progreso.
- Fase 2A (esqueleto hexagonal): proyecto Flutter Android; core, puertos y casos de uso PublishNeed/FlushOutbox; Drift; adaptadores locales; DI con Riverpod; router de 5 pestanas con go_router; pantalla de recuperacion; 9 pruebas.
- Fase 2B (recorrido principal): ubicacion, Explorar cerca con filtro/orden y Visto Bueno con regla anti-inflado, pedir en 3 toques o menos con frases, propuestas, aceptar con codigo, validar, servicio hecho, calificar, reputacion antes-despues; Drift esquema 2 con migracion probada; APK debug compila; 27 pruebas pasan.
- Modulos en lib/features: deals_chat, directory, emergency, explore, location, needs, proposals, reputation, settings, solidarity, sync.
- Dependencias: flutter_riverpod, go_router, drift, drift_flutter, uuid, intl (y flutter_map segun el registro de logs).
- Pendiente: Fase 3 (mejoras de UI/UX), Fase 4 (backend y mejoras, rama prevista mobile-mejoras). No probado en un telefono real.

6.3 Comparacion

Aspecto	feat/flutter-mobile	mobile-ux
Servidor	Supabase real (anonimo + codigo)	Ninguno: SQLite local con Drift
Arquitectura	Fina: api + store + pantallas	Hexagonal: puertos, casos de uso, adaptadores
Offline	Solo mientras la app sigue abierta	Cola de pendientes persistente e idempotente
Emergencia / GPS	No	Si (modulos emergency y location)
Pruebas	Una prueba generada, sin valor	27 pruebas (arquitectura, cola, migracion)
Multi-usuario real	Si, vecinos del piloto	No (contraparte de demostracion marcada)

7. Plan de 48 horas y equipo

Corte interno de features: domingo 8, 12:00 p. m. Despues solo se corrige. Entrega final: domingo 8, 6:30 p. m.

Rol	Quien	Dueno de
Lider / integracion	Jorge	Decisiones, merge final, demo, Supabase
Datos y backend	P1	Esquema, RLS, sync, Edge Function de Matrix
UI y flujos	P2	Inicio, Vecinos, Publicar, Perfil
Chat e infra	P3	Synapse, Caddy, transporte Matrix, APK
QA y guion	P4	Casos de prueba, accesibilidad, guion, backups

Fases del PLAN.md: F0 (noche del sabado: recorrer la app, decidir Supabase o PocketBase, subir a GitHub); F1 (domingo 6-9: servidor, sync con outbox idempotente, autenticacion minima, Matrix real); F2 (9-12, corte: dos telefonos reales, offline con modo avion, emergencia visible, trato completo con calificacion de dos vias); luego APK, deploy web y ensayo.

Decisiones tecnicas clave (docs/DECISIONES.md): SQLite local primero; Supabase como fuente de verdad; chat Matrix con wrapper REST propio; un room por trato; sin cifrado extremo a extremo en la demo; codigo de encuentro hasheado; ubicacion redondeada; sin router (navegacion por estado); sin pagos, push ni verificacion de identidad.

Guion de demo (4 actos, 3 minutos)

- **Acto 1, necesidad cotidiana:** Explorar cerca, Pedir este servicio, Publicar; tres propuestas; elegir, codigo de encuentro, resolver y calificar.
- **Acto 2, cadena de favores:** pedir un taladro prestado; oferta gratuita, trueque o alquiler; entrega y devolucion.
- **Acto 3, la red conoce capacidades:** Perfil, "Tu ayuda al barrio": evidencia, no privilegios.
- **Acto 4, emergencia y sin internet:** modo emergencia (institucion primero), modo avion, aviso pendiente, reabrir y reconectar.
- Frase de cierre: lo cotidiano construye la red que responde cuando importa.

Que decir si preguntan: real es GPS, offline, datos guardados e instalacion; simulado son vecinos, precios, respuestas y verificacion de identidad. No mostrar salvo que pregunten: ajuste de precio, ventas, alto contraste, chat con frases rapidas.

8. Riesgos principales (del registro de mobile-ux)

Los 45 riesgos completos estan en docs/03-riesgos.md; estos son los de mayor severidad.

Sev.	ID	Riesgo	Mitigacion
12	R2	Offline aparente: la sincronizacion del PWA es un setTimeout sin servidor	Cola persistente con UUID; no mostrar "Sincronizado" sin servidor
12	R17	Datos simulados presentados como hechos ("Recomendado por 14 vecinos")	Etiquetar como ejemplo; reescribir textos
12	R19	Arranque en frio: sin vecinos, la app se ve vacia	Seed marcado como demostracion; estados vacios guiados
12	R25	Inflado del Visto Bueno con el boton Recomendar (voto local, repetible)	Recomendar solo tras interaccion confirmada, una vez por persona
12	R26	Quechua sin revision nativa y cobertura parcial	Marcarlo como no revisado o retirarlo de la demo
12	R27	Nunca probado en telefono real ni con HTTPS publicado	Probar en Android real desde la primera version instalable
12	R1	Alcance excesivo (~30 pantallas; ~20 % de codigo sin uso)	Migrar por prioridad de demo
9	R30	Las "pruebas automaticas" del informe no estan en el repositorio	Reconstruir paridad desde el codigo
9	R31	El contador de 3 clics no cuenta el toque de abrir: publicar son 4 toques reales	Redisenar atajos; presentar "2 toques con atajo"
9	R32	Respuestas falsas en emergencia ("Marco Rivas quiere ayudarte")	Etiqueta SIMULACION visible; no mostrar como real
8	R10	Numeros de emergencia no verificados y con enlaces tel: que marcan de verdad	Verificar con fuentes oficiales; "por verificar" visible
8	R7	Exposicion de ubicacion exacta	Zona aproximada para terceros; exacta solo a quien acepto

Otros hallazgos de la lectura tecnica

- En app/src de main conviven dos implementaciones a medias: main.tsx usa StoreProvider (state/store, carpeta screens/ y ui/), pero App.tsx usa AppProvider (state/app, carpeta pages/ y components/). Hay que elegir una y borrar la otra.
- VITE_MATRIX_TOKEN se compila dentro del paquete web: cualquiera puede leerlo.
- El codigo de encuentro tiene solo 9.000 combinaciones: el servidor debe limitar intentos.
- Hay package-lock.json y pnpm-lock.yaml a la vez; los iconos y sw.js estan duplicados en la raiz y en app/public.
- El service worker exige subir la version del cache en cada publicacion; es facil olvidarlo.

9. Estado actual y proximos pasos

Hecho y verificado en esta sesion

- Clonado el repositorio de Jorge (todas las ramas) y leidos sus documentos y codigo clave.
- PWA/app React de main: npm install, tsc sin errores y servidor Vite corriendo; la app abre con los datos de ejemplo (probado en vista movil). El servidor sigue en segundo plano en http://127.0.0.1:5173.
- feat/flutter-mobile: android/ generado, flutter analyze lib sin errores, APK de depuracion instalado en el emulador Android 14 y abierto en la pantalla de entrada.

No verificado

- Ningun flujo detras de la entrada de Flutter (requiere el codigo del piloto).
- mobile-ux no se ejecuto en esta sesion (solo se leyeron sus documentos y estructura).
- Nada se probo en un telefono real, ni con HTTPS publicado, ni con dos telefonos a la vez.
- Los numeros de emergencia siguen sin verificarse contra fuentes oficiales.

Decisiones abiertas del equipo

- Que linea movil se presenta: feat/flutter-mobile (con servidor real) o mobile-ux (arquitectura limpia, sin servidor), o una fusion.
- Quien verifica los numeros de emergencia y para cuando (Q-5).
- Si el nivel solidario es publico o solo un distintivo (Q-6).
- Aceptar la regla unica anti-inflado del Visto Bueno (Q-3) y la interpretacion estricta de "3 clics" (Q-2).
- Revision legal de datos personales (Ley 29733) antes de abrir el piloto a mas vecinos.

Siguientes pasos sugeridos

- Probar el recorrido completo de la linea elegida en un Android real con HTTPS.
- Resolver la duplicacion de pages/ y screens/ en app/src antes de seguir desarrollando.
- Anadir limite de intentos al codigo de encuentro y quitar el token de Matrix del cliente.
- Etiquetar todo dato de ejemplo como demostracion y revisar el quechua con un hablante nativo.
- Ensayar el guion de 3 minutos evitando lo simulado.